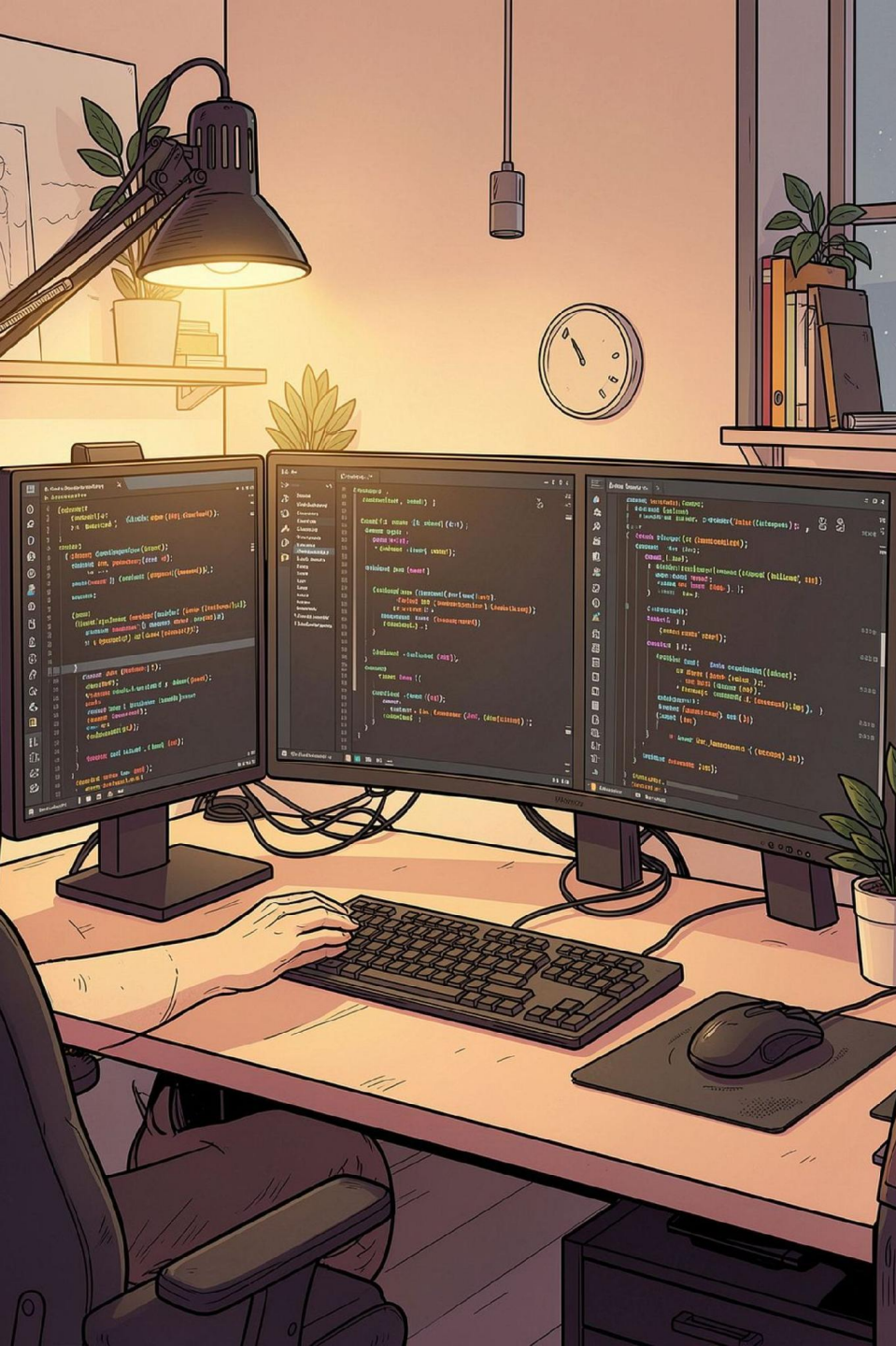




Engenharia Limpa: Clean Code & Clean Architecture

Disciplina 02 — Engenharia de Software
Geração Tech · Fullcycle

Por Que Engenharia Limpa Importa?

A qualidade do código não é apenas uma preferência estética — ela tem impacto direto na velocidade, custo e sustentabilidade de qualquer projeto de software.

80% em Manutenção

A maior parte do tempo em software não é gasta criando — é gasta **corrigindo e mantendo** o que já existe.

- Leitura de código consome mais tempo que a escrita
- Código difícil de entender gera bugs ao ser alterado
- Times crescem e novas pessoas precisam entender o sistema
- Custo de manutenção supera o custo de desenvolvimento original

Menos Bugs, Mais Agilidade

Código limpo reduz erros, facilita mudanças e economiza tempo real da equipe.

- Funções pequenas e coesas são mais fáceis de testar
- Nomes claros eliminam ambiguidade e mal-entendidos
- Menor acoplamento = menos efeitos colaterais inesperados
- Refatoração segura com testes bem estruturados

Crescimento Sustentável

Arquitetura limpa permite que o projeto **cresça sem virar "spaghetti"**.

- Separação de responsabilidades facilita evolução modular
- Novas funcionalidades não quebram as existentes
- Onboarding mais rápido de novos desenvolvedores
- Decisões técnicas reversíveis e com menor custo

Impacto no Negócio

Engenharia de baixa qualidade não é apenas um problema técnico — é um **risco para o negócio**.

- Dívida técnica acumulada reduz velocidade de entrega
- Sistemas frágeis aumentam o tempo de inatividade
- Reescritas completas são caras e arriscadas
- Times desmotivados com código de baixa qualidade produzem menos

📌 **Regra do Escoteiro:** "Deixe o código mais limpo do que você encontrou." — Robert C. Martin (Uncle Bob)

O Que É Engenharia Limpa?

CLEAN CODE

Código que Humanos Conseguem Ler

Código limpo é **legível, simples e expressivo**. Qualquer desenvolvedor do time consegue entender sem precisar de comentários ou de um guia.

Nomes Significativos

Variáveis e funções com nomes que revelam intenção:
`calcularTotalPedido()` em vez de `calc()`.

Funções Pequenas

Cada função faz **uma coisa só** e faz bem. Máximo de 20 linhas como regra geral.

Sem Duplicação

Princípio DRY — *Don't Repeat Yourself*. Lógica repetida vira fonte de bugs e inconsistências.

Comentários Honestos

Bom código se explica sozinho. Comentários devem explicar o **porquê**, nunca o **o quê**.

CLEAN ARCHITECTURE

Estrutura que Sobrevive ao Tempo

Clean Architecture organiza o sistema em **camadas com responsabilidades bem definidas**, separando regras de negócio de detalhes técnicos como banco de dados e frameworks.

Separação de Camadas

Entidades, casos de uso, interfaces e infraestrutura — cada camada conhece apenas as camadas internas.

Independência de Frameworks

O núcleo do sistema **não depende do Rails, Django ou Spring**. Frameworks são detalhes substituíveis.

Testabilidade Total

Regras de negócio testadas sem banco de dados, sem HTTP, sem dependências externas.

Inversão de Dependência

Módulos de alto nível não dependem de módulos de baixo nível — ambos dependem de **abstrações**.

📌 **Juntas:** Clean Code cuida da *legibilidade linha a linha*; Clean Architecture cuida da *estrutura do sistema como um todo*.

Princípios de Código Limpo

Escrever código que funciona é apenas o começo. O verdadeiro desafio é escrever código que **outras pessoas — e você mesmo no futuro — consigam entender, modificar e expandir com confiança**.

Neste capítulo, vamos explorar os princípios fundamentais que transformam código funcional em código verdadeiramente limpo. Esses princípios não são regras arbitrárias: eles emergem de décadas de experiência coletiva da indústria de software, sintetizadas por engenheiros que viram sistemas crescerem, encolherem e desmoronarem.

Nomenclatura Significativa

Como escolher nomes que revelam intenção e eliminam a necessidade de comentários explicativos.

Funções Pequenas e Focadas

Por que funções devem fazer uma coisa só — e como isso reduz bugs e facilita testes.

Comentários com Propósito

Quando comentar é um sinal de alerta, e quando é absolutamente necessário.

Formatação e Consistência

Como a organização visual do código comunica estrutura e respeito pelo time.

Ao final deste capítulo, você terá um conjunto de critérios claros para avaliar e melhorar qualquer trecho de código — seja código legado, código de terceiros, ou o seu próprio.

Nomes Significativos

✗ Ruim

```
let d = new  
Date();
```

✓ Bom

```
let dataAtual =  
new Date();
```

Por que isso importa?

Nomes claros **eliminam a necessidade de comentários explicativos**. O código se torna auto-documentado.

- Use nomes que revelem a intenção
- Evite abreviações obscuras
- Seja consistente no vocabulário do projeto

Funções Pequenas e Focadas



Uma Responsabilidade

Cada função faz **exatamente uma coisa**. Se precisar de "e" para descrevê-la, divida-a.



Fácil de Testar

Funções pequenas são **simples de testar e reutilizar** em qualquer parte do sistema.



Tamanho Ideal

20–30 linhas por função. Acima disso, avalie responsabilidades extras para extrair.

Tratamento de Erros

Um bom tratamento de erros é fundamental para código limpo, confiável e fácil de manter. Siga estas diretrizes para lidar com falhas de forma expressiva e controlada.

1 Use Exceções, Não Códigos de Erro

Retornar códigos de erro força o chamador a verificar o retorno imediatamente, poluindo o fluxo principal com lógica de controle.

Exceções são mais expressivas e permitem separar o caminho feliz do tratamento de falhas.

- Prefira exceções tipadas e semânticas (ex: `InvalidOrderException`)
- Evite `return null` ou `return -1` para sinalizar erros
- Use hierarquias de exceções para diferenciar erros recuperáveis de fatais
- Documente as exceções que cada função pode lançar

2 Trate Perto da Origem

Lide com o erro onde ele ocorre — não deixe propagar desnecessariamente pela aplicação. Quanto mais longe o erro viaja, mais difícil é entender seu contexto original e depurá-lo corretamente.

- Capture exceções no nível mais próximo onde você tem informação suficiente para tratá-las
- Evite blocos `catch` genéricos que silenciam erros silenciosamente
- Se não puder tratar, re-lance com contexto adicional (*exception chaining*)
- Use `finally` ou padrões RAII para liberar recursos independentemente do resultado

3 Mensagens Claras e Acionáveis

Erros devem dizer **o que aconteceu, onde e por quê** — úteis para debug e para o usuário. Uma mensagem vaga como *"Erro inesperado"* não ajuda ninguém a resolver o problema.

- Inclua o contexto: qual operação falhou e quais dados estavam envolvidos
- Diferencie erros técnicos (para logs) de mensagens amigáveis (para o usuário)
- Evite expor detalhes internos (stack traces, SQL) para o usuário final
- Registre erros com nível adequado: `WARN` para recuperáveis, `ERROR` para críticos

❏ Nunca ignore uma exceção silenciosamente com um bloco `catch` vazio. Se decidir não tratar, registre o erro com log e justifique no código.

Comentários: Quando Usar?

Evite

Comentários que explicam código **óbvio**. Se o código precisa de explicação, reescreva-o.

Use

Comentários para o "**por quê**", nunca para o "o quê". Contexto de decisão, não descrição de sintaxe.

"Código limpo é auto-explicativo. O melhor comentário é o código que não precisa de um."

Quando você sente necessidade de comentar, pergunte primeiro: **posso melhorar o nome desta função ou variável?**

Arquitetura Limpa

Arquitetura Limpa (*Clean Architecture*) é um conjunto de princípios de design de software proposto por Robert C. Martin ("Uncle Bob") que organiza o código em camadas concêntricas com responsabilidades bem definidas. No centro estão as regras de negócio — a essência do sistema — e nas camadas externas ficam os detalhes de infraestrutura: frameworks, bancos de dados, interfaces de usuário e APIs externas.

A ideia fundamental é simples, mas poderosa: **os detalhes devem depender das regras de negócio, nunca o contrário**. Isso significa que seu sistema não deve ser escravo de um framework, de um banco de dados ou de qualquer tecnologia específica. Se amanhã você precisar trocar o banco de dados, mudar o framework web ou adaptar o sistema para um novo canal, as regras de negócio permanecem intactas.

Por que isso importa?

Sistemas sem arquitetura bem definida tendem a acumular dívida técnica rapidamente. Com o tempo, a lógica de negócio se mistura com código de infraestrutura, os testes se tornam frágeis ou impossíveis, e cada mudança gera efeitos colaterais inesperados. O custo de evolução cresce enquanto a velocidade de entrega cai.

**Alto Acoplamento**
Mudanças em um framework ou banco de dados quebram a lógica de negócio.

**Dificuldade de Testar**
Sem separação de camadas, testar regras de negócio exige infraestrutura real.

**Evolução Lenta**
O sistema se torna rígido; cada nova funcionalidade é custosa e arriscada.

O que a Arquitetura Limpa resolve?

Ao estabelecer fronteiras claras entre camadas e definir a direção das dependências, a Arquitetura Limpa torna o sistema **testável** (as regras de negócio podem ser testadas sem banco de dados ou UI), **evolutivo** (detalhes de infraestrutura podem ser trocados com impacto mínimo) e **independente de frameworks** (o framework é uma ferramenta, não o alicerce do sistema).

O que veremos neste capítulo

01	02
As camadas da arquitetura Entidades, Casos de Uso, Adaptadores de Interface e Frameworks — o que cada camada faz e onde cada tipo de código pertence.	A Regra de Dependência O princípio central que governa como as camadas se comunicam e por que as dependências só podem apontar para dentro.
03	04
Fronteiras e contratos Como definir interfaces que isolam as camadas e permitem substituir implementações sem alterar o núcleo do sistema.	Aplicação prática Exemplos reais de como estruturar projetos, organizar pacotes e tomar decisões de design alinhadas com esses princípios.

As 4 Camadas da Arquitetura Limpa

As dependências apontam sempre para dentro: das ferramentas externas em direção ao núcleo de regras de negócio.

1

Entidades (Entities)

Regras de negócio corporativas, totalmente independentes de frameworks ou bibliotecas externas.

2

Casos de Uso (Use Cases)

Lógica específica da aplicação. Organiza o fluxo de dados de e para as entidades.

3

Adaptadores de Interface (Interface Adapters)

Controladores, Gateways e Apresentadores. Converte dados do formato externo para o formato dos casos de uso.

4

Frameworks e Drivers (Frameworks & Drivers)

Web, Banco de Dados, UI e detalhes de implementação. A camada mais externa onde residem as ferramentas.

Conclusão: O Caminho do Desenvolvedor FullCycle

Transformando teoria em código limpo, testável e de alto impacto no Geração Tech.

CÓDIGO LIMPO

Simplicidade e Clareza

- **Funções Focadas:** Uma única responsabilidade e tamanho ideal (20–30 linhas).
- **Nomes Expressivos:** Variáveis e métodos que explicam a intenção sem rodeios.
- **Tratamento Consciente:** Exceções semânticas em vez de códigos de erro ou blocos vazios.
- **Comentários do "Porquê":** Código autoexplicativo com foco no contexto de decisão.

ARQUITETURA LIMPA

Isolamento e Evolução

- **4 Camadas Claras:** Entidades, Casos de Uso, Adaptadores e Drivers.
- **Regra de Dependência:** O fluxo aponta sempre para o núcleo do negócio.
- **Desacoplamento:** Regras de negócio totalmente independentes de frameworks e bancos de dados.
- **Contratos Firmes:** Interfaces bem definidas facilitam substituições de infraestrutura.

VISÃO FULLCYCLE

Sustentabilidade do Software

- **Testabilidade Total:** Testes unitários rápidos e confiáveis sem dependência externa.
- **Baixa Dívida Técnica:** Facilidade de manutenção e menor risco em alterações.
- **Entrega Sustentável:** Agilidade constante ao longo do ciclo de vida do produto.
- **Próximo Passo:** Aplicar os conceitos no projeto prático do Geração Tech.

 **Lembre-se:** "Código limpo não é sobre escrever menos código, é sobre tornar o futuro do projeto viável e simples de manter."